Demonstrating BriskState: A Cache-Aware State Backend for Apache Flink

Anonymous Authors
Anonymous Submission
Anonymous, Anonymous

Abstract

State-intensive stream processing often becomes memory-bound before it becomes compute-bound. Frequent state lookups, window maintenance, and timer firing amplify cache misses, long object-indirection chains, and heap-management overhead, making state-backend efficiency a first-order performance concern. We present BriskState, an Apache Flink state backend that combines a thin Java control layer, a JNI bridge, and a native execution engine to reorganize keyed state around compact, locality-aware structures. This demo shows how these design choices improve over heap-based state backends on the official Flink state-backend JMH suite and on a seven-workload benchmark set. The demo supports side-by-side execution and compact runtime summaries so that throughput, tail-latency, and hot-state differences can be tied back to backend organization under identical workloads.

Keywords

stream processing, state backend, BriskState, cache-aware data layout, Apache Flink

1 Introduction

Large-state stream-processing pipelines rely on the state backend on nearly every record. In Apache Flink, this path is exercised by point updates, window aggregation, joins, and timer callbacks, so backend behavior can dominate end-to-end performance even when user logic is lightweight [6]. In practice, the bottleneck is frequently the memory hierarchy rather than arithmetic throughput: state accesses trigger pointer chasing, touch multiple cache lines, and create runtime jitter through heap-management activity.

This is directly relevant to parallel processing because modern streaming jobs scale out compute across many task slots while repeatedly converging on shared bottlenecks in the memory hierarchy. When each parallel task maintains large keyed state, poor locality amplifies cache and TLB pressure, introduces tail-latency spikes, and makes additional cores less effective than their nominal count suggests. A backend that preserves locality on the state path therefore improves not just raw throughput, but also how well a multicore deployment sustains parallel speedups under skewed and state-heavy workloads.

We present BriskState, a cache-aware Flink state backend designed to improve state-access locality in state-intensive workloads.

BriskState keeps Flink integration in a thin Java control layer and moves keyed-state lookup, routing, and storage into a native engine reached through JNI. The demo compares BriskState against heap-based backends on identical workloads and highlights how compact state layout and locality-aware access paths affect throughput, tail latency, and hot-state behavior. Rather than presenting only final summary numbers, we focus on the mechanism behind those numbers: how the backend partitions state by key-group and namespace, how typed JNI calls enter the native path, and how a compact table layout changes the cost profile of frequent updates and lookups.

BriskState is positioned against three nearby lines of work. Stateful dataflow systems such as MillWheel, Flink, Naiad, Structured Streaming, and the Dataflow model define high-level execution semantics, but the physical organization of state remains largely hidden behind operator APIs and runtime services [2–4, 6, 9]. In Flink deployments, backend choices usually reduce to in-memory heap structures or embedded persistent stores such as RocksDB, while work on consistent state management and migration, including Flink state management and Megaphone, emphasizes correctness and continuity rather than layout-aware backend optimization [5, 6, 8, 10]. High-performance key-value engines and compact hash-table designs, including FASTER and Swiss-table-style indexing, show that memory layout and probing discipline materially change access cost, while application mixes such as the seven topologies revisited by Zhang et al. expose how operator structure interacts with modern multicore hardware [1, 7, 11]. BriskState targets this gap by implementing a Flink-compatible state backend whose layout-driven optimizations deliver measurable gains over heap-based alternatives on both microbenchmark and application workloads.

This paper makes three contributions:

- It presents BriskState as a Flink-compatible cache-aware state backend built from a Java control layer, a JNI bridge, and a native execution engine.
- It demonstrates, using the official Flink state-backend microbenchmark and a seven-workload benchset, that BriskState improves throughput and tail latency over standard heap-based baselines.
- It provides a compact demonstration methodology that links observed gains to key-group routing, namespace partitioning, and compact native storage.

2 BriskState

2.1 Overview

Figure 1 summarizes the implemented BriskState architecture together with the companion demo UI used in the live presentation. The left side shows the execution stack from

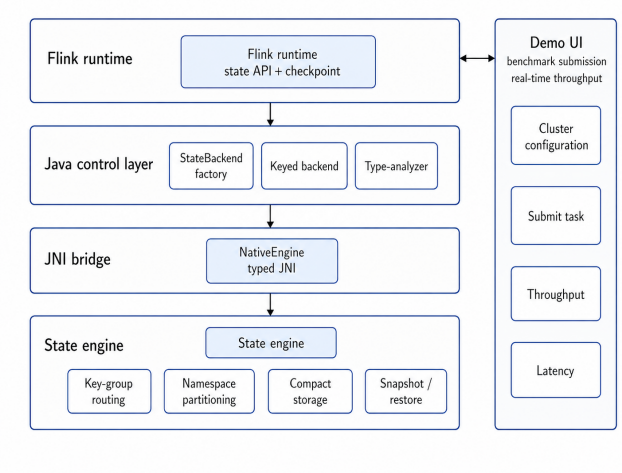

Figure 1: BriskState architecture and the companion demo UI.

Flink runtime to the state engine; the right side shows the demo interface used for benchmark submission, cluster configuration, and live throughput and latency display.

The figure distinguishes the execution stack from the presentation layer. Flink integration and serializer analysis stay in Java, while keyed-state storage, namespace partitioning, and snapshot consistency move into the state engine. The demo UI is shown separately because it is used to configure the cluster, submit benchmark runs, and display live throughput and latency during the presentation. The underlying storage path follows compact, cache-conscious hash-table ideas inspired by modern Swiss-table designs [1], and those choices are precisely what drive BriskState's gains over heap-based baselines.

That systems view maps cleanly to the implemented storage hierarchy. At the backend level, state is partitioned first by key-group and then by namespace before it reaches a compact sub-table. When the namespace is the common void namespace used by many keyed operators, the extra namespace map can be skipped and the backend addresses one table per key-group directly; otherwise each key-group owns a namespace-to-table map, and empty namespaces are removed eagerly after clears. This design reduces routing overhead on the hot path while keeping state isolation explicit enough to visualize per-namespace skew during the demonstration.

BriskState is evaluated with three workload families. The first is a WordCount-style stress test that isolates high-frequency state updates. The second is the official Flink `flink-state-benchmark` JMH suite for state backends, which isolates per-operation ValueState, ListState, and MapState costs under controlled loops. The third is our own *benchset*: a seven-application benchmark set, adapted from the task mix in Zhang et al. [11], that combines fan-out, fan-in, and branch-merge topologies to stress state routing under more application-like pipelines. Together, they show where BriskState's backend design pays off, from isolated state operations to end-to-end streaming jobs.

2.2 Evaluation Setup

BriskState is evaluated with Flink-compatible state management, the official Flink state-backend JMH suite, and the WordCount and benchset jobs used for end-to-end comparison. The evaluation therefore builds on real workloads and measured results rather than on a synthetic front end alone.

Table 1 summarizes the experimental platform used for both the microbenchmark and the end-to-end Flink runs.

BriskState also remains faster at the operator level. Across the 13 retained `flink-state-benchmark` operations, the largest gains appear on list iteration (+132%), map insertion (+110%), map lookup (+91%), value insertion (+87%), and map removal (+74%). These measurements come from the official Flink state-backend JMH benchmark rather than a separate operator-level application benchmark, so they isolate the state-access path directly.

For the live demonstration, those measurements serve two purposes. First, they give the audience a stable microbenchmark reference before moving to larger job graphs. Second, they connect end-to-end pipeline behavior back to concrete state operations such as point update, map access, and list traversal.

The demo is organized in four stages:

(1) select a workload and switch between heap-based and cache-aware backends;
(2) compare time-series changes in throughput, tail latency, and backend-specific counters;
(3) examine which key-groups and namespaces become hot and how occupancy evolves;
(4) compare metrics and state summaries across repeated runs.

The demo focuses on how BriskState's layout decisions translate into externally visible gains over heap-based backends.

Our own benchset complements the state-backend microbenchmark with seven application kernels: Stateful Word Count (WC), Fraud Detection (FD), Spike Detection (SD), Traffic Monitoring (TM), Log Processing (LG), Spam Detection in VoIP (VS), and Linear Road (LR). Adapted from the topology mix in Zhang et al. [11], it instantiates those jobs so their stateful operators exercise BriskState under the same Flink deployment. Figure 3 shows 1.11×–1.50× higher throughput per core on six workloads, with parity on VS.

2.3 Runtime Support

The comparison interface is intentionally lightweight. Metrics are sampled at coarse intervals from the workload drivers and Flink runtime, while the state view focuses on summaries such as namespace occupancy and key-group heat.

The backend implementation also contains details that matter for interpretation. Within each sub-table, control

Table 1: Experimental setup.

Item	Configuration
Hardware platform	Intel Xeon Platinum 8368Q x86_64 server, 38 physical cores / 76 hardware threads, 251 GiB memory
Software stack	Linux, Apache Flink, and the implemented BriskState backend
Flink deployment	One JobManager and two TaskManagers for controlled side-by-side backend comparison
TaskManager config	Four slots per TaskManager with 16 GiB process memory per TaskManager
Default parallelism	8

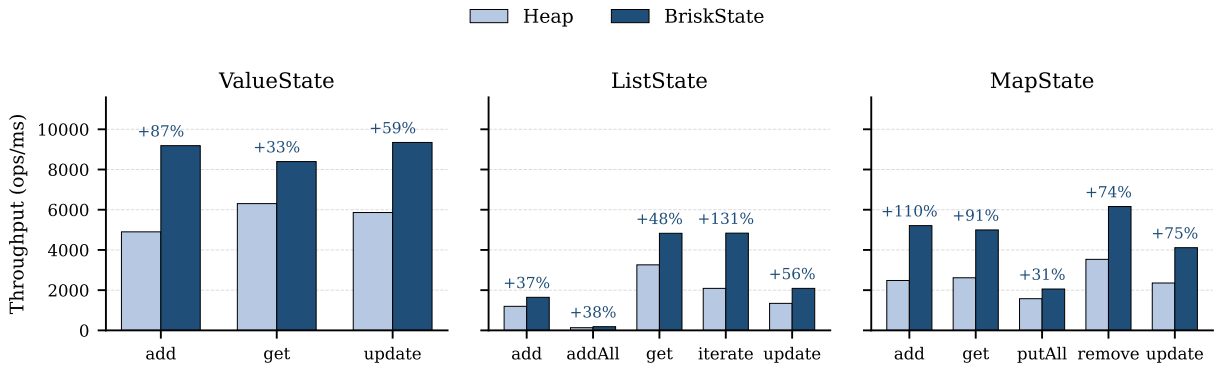

Figure 2: Results from the official Flink `flink-state-benchmark` JMH suite on our local experiment setup. BriskState outperforms the heap backend across retained ValueState, ListState, and MapState operations.

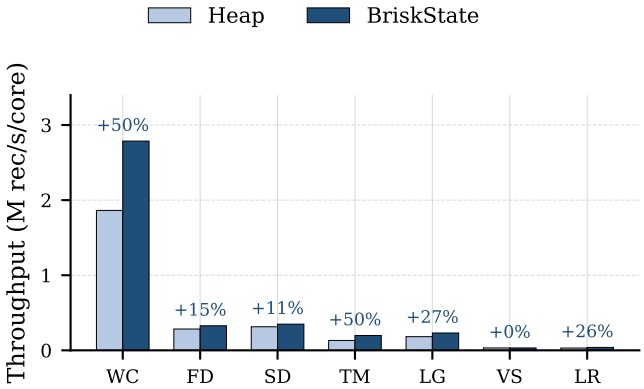

Figure 3: Throughput per core on our benchset. BriskState improves six of the seven application pipelines and matches the heap baseline on the VoIP spam-detection workload.

metadata and payload arrays are organized to support compact probing and linear snapshot traversal, and the implementation includes specialized primitive-key paths that avoid boxing for common integer and long keys. These choices help explain why BriskState reduces routing overhead and stabilizes state-access cost relative to heap-based baselines.

The comparison supports live execution together with repeated runs. Long-running queries and hardware-counter collection can be sensitive to machine load and deployment conditions, so the evaluation keeps the reported metrics and structural summaries in a common format across runs. This preserves comparability while keeping the emphasis on BriskState's relative gains. It also makes the demo practical in a conference setting: the operator can restart a workload, switch the backend, and immediately present the same set of counters, plots, and state summaries without changing the surrounding workflow.

BriskState is designed to make backend choices and their performance consequences concrete. It ties state-layout design and memory locality to measured throughput and latency gains in state-intensive Flink jobs, while still separating backend effects from operator logic.

3 Demonstration Scenario

The demonstration begins with a side-by-side comparison mode. One workload is executed with two different backends, and the synchronized dashboard reports throughput, latency percentiles, and backend-specific counters. This mode establishes that BriskState improves not only final job time but also runtime stability and responsiveness. During the live session, the presenter will start from WordCount because it gives visible throughput changes within a short time window and makes it easy for the audience to see how backend choice alone alters the steady-state execution profile.

The second stage connects the optimization back to state layout directly. The interface renders a hierarchical view from operator to key-group to namespace, then overlays heat and occupancy information to show where accesses concentrate. This view shows how a smaller and more stable working set appears when namespaces are separated into compact subtables, and how that change corresponds to the throughput and latency gains seen in comparison runs. At this point,

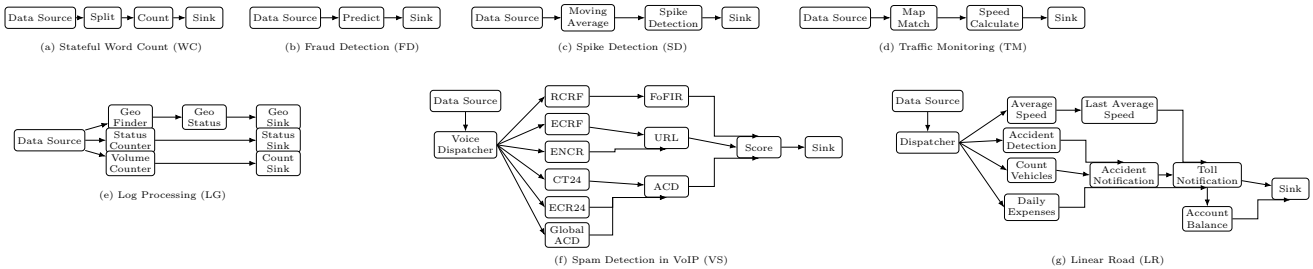

Figure 4: Benchset topologies adapted from Zhang et al. [11]. The set spans seven stateful pipelines with different branch, merge, and routing structure.

audience members can request a workload switch or ask to inspect a specific hot key-group, allowing the demo to move from aggregate metrics to a structural explanation of where the saved work comes from.

The third stage steps through three workload families under the same dashboard: WordCount for pure update stress, `flink-state-benchmark` for per-operation state costs, and benchset for application pipelines with richer branching and merging structure. The presenter will use this sequence to answer three increasingly demanding questions: whether the backend accelerates a simple state-update loop, whether the gain persists on isolated Flink state operations, and whether the same trend survives in multi-operator pipelines with branch and merge structure.

Figure 4 explains why benchset is more informative than a single monolithic query: some pipelines are short and update-dense, some branch into multiple keyed paths, and others merge several stateful summaries before the sink.

The audience interaction is intentionally lightweight but concrete. Attendees can choose which workload family to launch next, ask for a rerun under the heap baseline, and compare the resulting throughput-per-core and latency summaries against the previous run. Because the same scripts and deployment layout are reused throughout the session, the interaction remains focused on backend behavior rather than on one-off tuning. The demo therefore communicates not only that BriskState is faster on these Intel-hosted experiments, but also why the improvement is reproducible across operator-level and application-level views.

4 Discussion and Conclusion

State backends are central to modern stream processing, but their most important properties are physical rather than logical. BriskState turns those physical choices into measurable gains by combining compact native storage, locality-aware access paths, and a Flink-compatible integration layer. The resulting demonstration makes it possible to connect backend design directly to throughput and latency improvements over heap-based backends.

The demonstration combines the native state backend, the evaluation workloads, workload control, and synchronized metrics to compare BriskState directly against heap-based baselines under the same workloads. It shows that state layout materially affects stream-processing performance and stability.

References

[1] Abseil. 2018. Swiss Tables Design Notes. https://abseil.io/about/design/swisstables.

[2] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1033–1044.

[3] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.

[4] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2018. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data*. 601–613.

[5] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State Management in Apache Flink®: Consistent Stateful Distributed Stream Processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.

[6] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* 38, 4 (2015), 28–38.

[7] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 International Conference on Management of Data*. 275–290.

[8] Hyeontaek Lim Hoffman, Ryan Stutsman, Frank McSherry, and Malte Schwarzkopf. 2019. Megaphone: Latency-Conscious State Migration for Distributed Streaming Dataflows. In *Proceedings of the 2019 International Conference on Management of Data*. 435–450.

[9] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. 2013. Naiad: A Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 439–455.

[10] RocksDB Project. 2024. RocksDB. https://rocksdb.org/. Accessed 2026-04-24.

[11] Shuhao Zhang, Bingsheng He, Daniel Dahlmeier, Amelie Chi Zhou, and Thomas Heinze. 2017. Revisiting the Design of Data Stream Processing Systems on Multi-Core Processors. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. doi:10.1109/ICDE.2017.119